합의 알고리즘 검증 분석

개념적 배경

왜 합의 알고리즘이 필요한가?

합의 알고리즘은 분산 네트워크에서 모든 노드가 동일한 블록체인 상태에 합의하도록 보장하는 핵심 메커니즘입니다. Ethereum은 다음과 같은 이유로 합의 알고리즘이 필요합니다:

1. **분산 합의**: 중앙 권한 없이 네트워크 참여자들이 블록체인 상태에 합의
2. **이중 지불 방지**: 동일한 자산이 두 번 사용되는 것을 방지
3. **네트워크 보안**: 악의적인 공격자로부터 네트워크 보호
4. **최종성**: 트랜잭션이 되돌릴 수 없음을 보장

Ethereum의 합의 알고리즘 진화

Ethereum은 Proof of Work(PoW)에서 Proof of Stake(PoS)로 전환했습니다:

- **Ethash (PoW)**: 2015-2022년, 계산 집약적 퍼즐 해결
- **Beacon Chain (PoS)**: 2022년 이후, 검증자의 지분 기반 합의

구현 분석

핵심 데이터 구조

1. 합의 엔진 인터페이스

```
type Engine interface {
```

```
Author(header *types.Header) (common.Address, error)

VerifyHeader(chain ChainHeaderReader, header *types.Header) error

VerifyHeaders(chain ChainHeaderReader, headers []*types.Header) (chan<- struct{}

VerifyUncles(chain ChainReader, block *types.Block) error

Prepare(chain ChainHeaderReader, header *types.Header) error

Finalize(chain ChainHeaderReader, header *types.Header, state vm.StateDB, body *

FinalizeAndAssemble(chain ChainHeaderReader, header *types.Header, state *state.

Seal(chain ChainHeaderReader, block *types.Block, results  chan<- *types.Block,

SealHash(header *types.Header) common.Hash

CalcDifficulty(chain ChainHeaderReader, time  uint64, parent *types.Header) *big

Close() error

}
```

2. Ethash (PoW) 구조체

```
type Ethash struct {

fakeFail *uint64  // 테스트용: 실패할 블록 번호

fakeDelay *time.Duration // 테스트용: 검증 지연 시간

fakeFull  bool  // 테스트용: 모든 블록 승인

}
```

3. Beacon (PoS) 구조체

```
type Beacon struct {
```

```
ethone consensus.Engine // 임베디드 eth1 합의 엔진 (ethash 또는 clique)
}
```

주요 알고리즘

1. Ethash 난이도 계산 (PoW)

```
func  makeDifficultyCalculator(bombDelay *big.Int) func(time  uint64, parent *ty
bombDelayFromParent := new(big.Int).Sub(bombDelay, big1)
return  func(time  uint64, parent *types.Header) *big.Int {
// 난이도 조정 공식:
// diff = (parent_diff +
// (parent_diff / 2048 * max((2 if len(parent.uncles) else 1) - ((timestamp - pa
// ) + 2^(periodCount - 2)

bigTime := new(big.Int).SetUint64(time)
bigParentTime := new(big.Int).SetUint64(parent.Time)

x := new(big.Int)
y := new(big.Int)

// (2 if len(parent_uncles) else 1) - (block_timestamp - parent_timestamp) // 9
x.Sub(bigTime, bigParentTime)
x.Div(x, big9)
```

```
if  parent.UncleHash == types.EmptyUncleHash {
x.Sub(big1, x)
} else {
x.Sub(big2, x)
}

// max(..., -99) 제한
if  x.Cmp(bigMinus99) < 0 {
x.Set(bigMinus99)
}

// parent_diff + (parent_diff / 2048 * adjustment_factor)
y.Div(parent.Difficulty, params.DifficultyBoundDivisor)
x.Mul(y, x)
x.Add(parent.Difficulty, x)

// 최소 난이도 보장
if  x.Cmp(params.MinimumDifficulty) < 0 {
x.Set(params.MinimumDifficulty)
}

// 아이스 에이지 (difficulty bomb) 계산
fakeBlockNumber := new(big.Int)
```

```
if parent.Number.Cmp(bombDelayFromParent) >= 0 {
fakeBlockNumber = fakeBlockNumber.Sub(parent.Number, bombDelayFromParent)
}


periodCount := fakeBlockNumber
periodCount.Div(periodCount, expDiffPeriod)


// 지수적 증가: diff = diff + 2^(periodCount - 2)
if periodCount.Cmp(big1) > 0 {
y.Sub(periodCount, big2)
y.Exp(big2, y, nil)
x.Add(x, y)
}
return x
}
}
```####
2. 블록 헤더 검증 (Ethash)


```go
func (ethash *Ethash) verifyHeader(chain consensus.ChainHeaderReader, header, pa
// Extra-data 크기 검증
if uint64(len(header.Extra)) > params.MaximumExtraDataSize {
```

```
return fmt.Errorf("extra-data too long: %d > %d", len(header.Extra), params.Max
}

// 타임스탬프 검증
if !uncle {
if header.Time > uint64(unixNow+allowedFutureBlockTimeSeconds) {
return consensus.ErrFutureBlock
}
}
if header.Time <= parent.Time {
return errOlderBlockTime
}

// 난이도 검증
expected := ethash.CalcDifficulty(chain, header.Time, parent)
if expected.Cmp(header.Difficulty) != 0 {
return fmt.Errorf("invalid difficulty: have %v, want %v", header.Difficulty, ex
}

// 가스 한도 검증
if header.GasLimit > params.MaxGasLimit {
return fmt.Errorf("invalid gasLimit: have %v, max %v", header.GasLimit, params.
}
```

```
if  header.GasUsed > header.GasLimit {
return  fmt.Errorf("invalid gasUsed: have %d, gasLimit %d", header.GasUsed, head
}

// 블록 번호 검증
if  diff := new(big.Int).Sub(header.Number, parent.Number);  diff.Cmp(big.NewInt
return  consensus.ErrInvalidNumber
}

// EIP-1559 검증
if  chain.Config().IsLondon(header.Number) {
if  err := eip1559.VerifyEIP1559Header(chain.Config(), parent, header);  err !=
return  err
}
}

return  nil
}
```

3. Beacon (PoS) 헤더 검증

```
func (beacon *Beacon) verifyHeader(chain consensus.ChainHeaderReader, header, pa
// Extra-data 크기 검증 (32바이트 제한)
```

```
if len(header.Extra) > int(params.MaximumExtraDataSize) {
return fmt.Errorf("extra-data longer than 32 bytes (%d)", len(header.Extra))
}

// PoS 특정 필드 검증
if header.Nonce != beaconNonce {
return errInvalidNonce
}
if header.UncleHash != types.EmptyUncleHash {
return errInvalidUncleHash
}

// 타임스탬프 검증 (미래 블록 허용)
if header.Time <= parent.Time {
return errInvalidTimestamp
}

// 난이도는 항상 0이어야 함
if beaconDifficulty.Cmp(header.Difficulty) != 0 {
return fmt.Errorf("invalid difficulty: have %v, want %v", header.Difficulty, be
}

// 가스 한도 검증
```

```
if  header.GasLimit > params.MaxGasLimit {
return  fmt.Errorf("invalid gasLimit: have %v, max %v", header.GasLimit, params.
}
if  header.GasUsed > header.GasLimit {
return  fmt.Errorf("invalid gasUsed: have %d, gasLimit %d", header.GasUsed, head
}

// 블록 번호 검증
if  diff := new(big.Int).Sub(header.Number, parent.Number);  diff.Cmp(common.Big
return  consensus.ErrInvalidNumber
}

// EIP-1559 검증
if  err := eip1559.VerifyEIP1559Header(chain.Config(), parent, header);  err !=
return  err
}

// Shanghai 포크 이후 withdrawalsHash 검증
shanghai := chain.Config().IsShanghai(header.Number, header.Time)
if  shanghai && header.WithdrawalsHash == nil {
return  errors.New("missing withdrawalsHash")
}
if !shanghai && header.WithdrawalsHash != nil {
```

```
return fmt.Errorf("invalid withdrawalsHash: have %x, expected nil", header.With
}

// Cancun 포크 관련 필드 검증
cancun := chain.Config().IsCancun(header.Number, header.Time)
if cancun {
if header.ParentBeaconRoot == nil {
return errors.New("header is missing beaconRoot")
}
if err := eip4844.VerifyEIP4844Header(chain.Config(), parent, header); err !=
return err
}
}

return nil
}
```

4. Uncle 블록 검증

```
func (ethash *Ethash) VerifyUncles(chain consensus.ChainReader, block *types.Blo
// Uncle 개수 제한 (최대 2개)
if len(block.Uncles()) > maxUncles {
return errTooManyUncles
```

```
}
if len(block.Uncles()) == 0 {
return nil
}


// 과거 7세대의 uncle과 조상 수집
uncles, ancestors := mapset.NewSet[common.Hash](), make(map[common.Hash]*types.H
number, parent := block.NumberU64()-1, block.ParentHash()
for i := 0; i < 7; i++ {
ancestorHeader := chain.GetHeader(parent, number)
if ancestorHeader == nil {
break
}
ancestors[parent] = ancestorHeader
if ancestorHeader.UncleHash != types.EmptyUncleHash {
ancestor := chain.GetBlock(parent, number)
if ancestor == nil {
break
}
for _, uncle := range ancestor.Uncles() {
uncles.Add(uncle.Hash())
}
}
```

```
parent, number = ancestorHeader.ParentHash, number-1
}

ancestors[block.Hash()] = block.Header()
uncles.Add(block.Hash())

// 각 uncle 검증
for _, uncle := range block.Uncles() {
hash := uncle.Hash()
// 중복 uncle 검사
if uncles.Contains(hash) {
return errDuplicateUncle
}
uncles.Add(hash)

// Uncle이 조상이 아닌지 검사
if ancestors[hash] != nil {
return errUncleIsAncestor
}
// Uncle의 부모가 유효한 조상인지 검사
if ancestors[uncle.ParentHash] == nil || uncle.ParentHash == block.ParentHash()
return errDanglingUncle
}
```

```
// Uncle 헤더 자체 검증
if err := ethash.verifyHeader(chain, uncle, ancestors[uncle.ParentHash], true,
return err
}
}
return nil
}
```

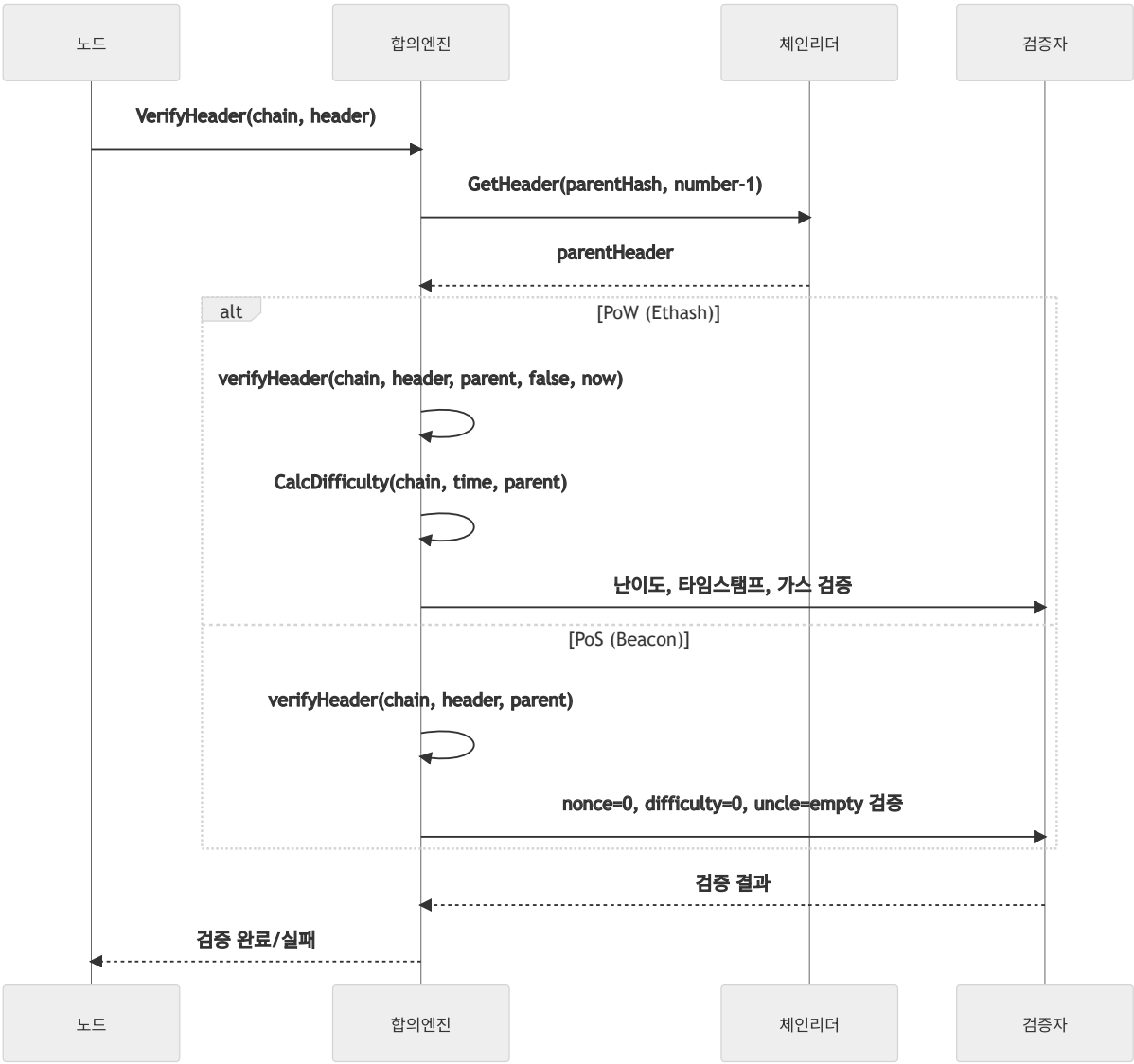
코드 구조 및 모듈 관계

```
consensus/
├── consensus.go # 합의 엔진 인터페이스
├── errors.go # 공통 에러 정의
├── ethash/ # Proof of Work 구현
| ├── consensus.go # PoW 합의 로직
| ├── difficulty.go # 난이도 계산
| └── ethash.go # Ethash 엔진
├── beacon/ # Proof of Stake 구현
| └── consensus.go # PoS 합의 로직
├── clique/ # Proof of Authority 구현
└── misc/ # 보조 검증 함수들
├── eip1559/ # EIP-1559 검증
```

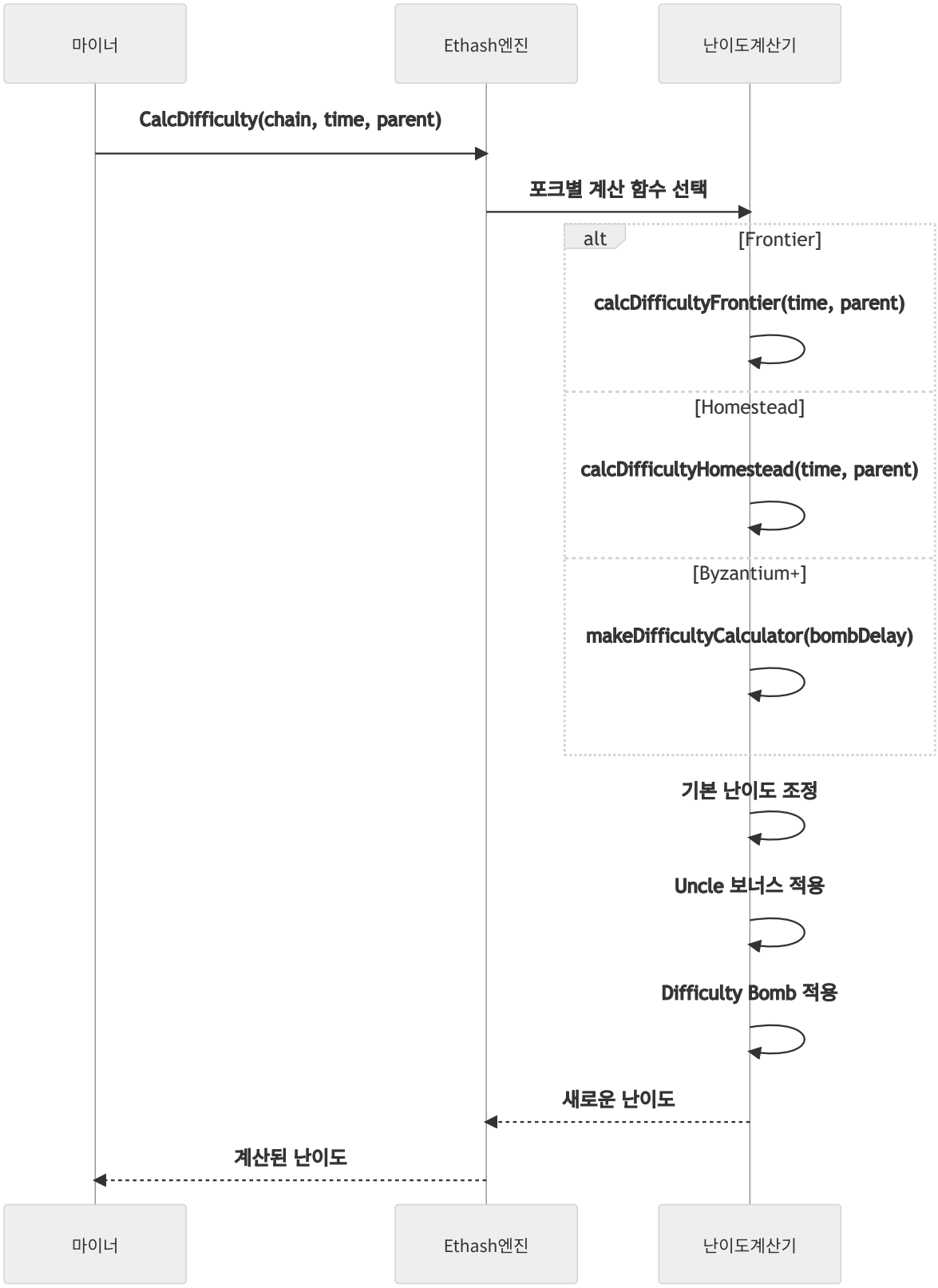
```
└── eip4844/ # EIP-4844 검증
```

실행 흐름

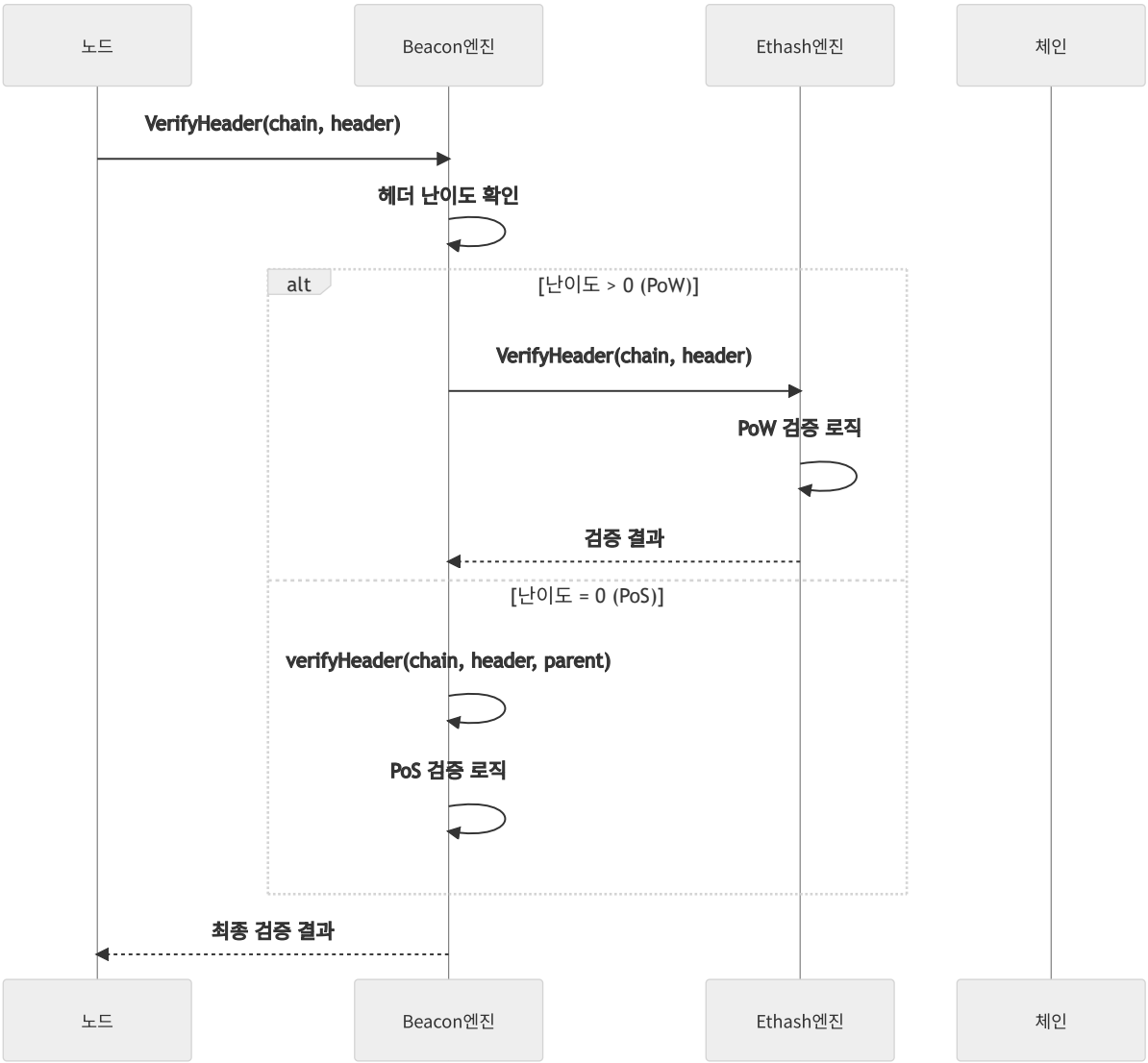
1. 블록 검증 과정



2. 난이도 조정 과정 (PoW)


마이너
Ethash엔진
난이도계산기
CalcDifficulty(chain, time, parent)
포크별 계산 함수 선택
alt
[Frontier]
calcDifficultyFrontier(time, parent)
[Homestead]
calcDifficultyHomestead(time, parent)
[Byzantium+]
makeDifficultyCalculator(bombDelay)
기본 난이도 조정
Uncle 보너스 적용
Difficulty Bomb 적용
새로운 난이도
계산된 난이도
마이너
Ethash엔진
난이도계산기

3. PoW에서 PoS 전환 과정



4. 단계별 실행 과정

블록 헤더 검증 단계:

1. **기본 검증**: Extra-data 크기, 타임스탬프, 블록 번호
2. **합의별 검증**:
   - PoW: 난이도, nonce, mixHash
   - PoS: 난이도=0, nonce=0, uncle=empty
3. **가스 검증**: 가스 한도, 가스 사용량
4. **포크별 검증**: EIP-1559, EIP-4844 등
5. **최종 승인**: 모든 검증 통과 시 블록 승인

난이도 계산 단계 (PoW):

1. **기본 조정**: 부모 블록 대비 시간 차이에 따른 조정
2. **Uncle 보너스**: Uncle 블록 포함 시 난이도 증가
3. **Difficulty Bomb**: 주기적 지수 증가
4. **최소값 보장**: 최소 난이도 이하로 떨어지지 않도록 보장
5. **포크별 조정**: 각 하드포크마다 다른 계산 방식

성능 최적화

1. 배치 검증

```
func (ethash *Ethash) VerifyHeaders(chain consensus.ChainHeaderReader, headers [

abort := make(chan  struct{})

results := make(chan  error, len(headers))

unixNow := time.Now().Unix()
```

```
go func() {
for i, header := range headers {
var parent *types.Header
if i == 0 {
parent = chain.GetHeader(headers[0].ParentHash, headers[0].Number.Uint64()-1)
} else if headers[i-1].Hash() == headers[i].ParentHash {
parent = headers[i-1]
}
var err error
if parent == nil {
err = consensus.ErrUnknownAncestor
} else {
err = ethash.verifyHeader(chain, header, parent, false, unixNow)
}
select {
case <-abort:
return
case results <- err:
}
}
}()
return abort, results
```

```
}
```

2. 캐싱 최적화

```
// 난이도 계산 결과 캐싱
var difficultyCache = make(map[common.Hash]*big.Int)
var cacheMutex sync.RWMutex

func cachedCalcDifficulty(config *params.ChainConfig, time uint64, parent *typ
key := crypto.Keccak256Hash(parent.Hash().Bytes(),
new(big.Int).SetUint64(time).Bytes())
cacheMutex.RLock()
if cached, ok := difficultyCache[key]; ok {
cacheMutex.RUnlock()
return new(big.Int).Set(cached)
}
cacheMutex.RUnlock()
difficulty := CalcDifficulty(config, time, parent)
cacheMutex.Lock()
difficultyCache[key] = new(big.Int).Set(difficulty)
cacheMutex.Unlock()
return difficulty
```

```
}
```

3. 병렬 검증

```
func (beacon *Beacon) VerifyHeaders(chain consensus.ChainHeaderReader, headers [

preHeaders, postHeaders := beacon.splitHeaders(headers)

if len(postHeaders) == 0 {

return beacon.ethone.VerifyHeaders(chain, headers)

}

if len(preHeaders) == 0 {

return beacon.verifyHeaders(chain, headers, nil)

}

// PoW와 PoS 헤더를 병렬로 검증

var (

abort = make(chan struct{})

results = make(chan error, len(headers))

)

go func() {

var (

old, new, out = 0, len(preHeaders), 0

errors = make([]error, len(headers))

done = make([]bool, len(headers))

oldDone, oldResult = beacon.ethone.VerifyHeaders(chain, preHeaders)
```

```
newDone, newResult = beacon.verifyHeaders(chain, postHeaders, preHeaders[len(pre
)
// 결과 수집 및 순서 보장
for {
for  ;  done[out];  out++ {
results <- errors[out]
if  out == len(headers)-1 {
return
}
}
select {
case  err := <-oldResult:
if !done[old] {
errors[old], done[old] = err, true
}
old++
case  err := <-newResult:
errors[new], done[new] = err, true
new++
case <-abort:
close(oldDone)
close(newDone)
return
```

```
}
}
}()
return abort, results
}
```

실제 사용 예시

1. 블록 검증

```
func validateBlock(engine consensus.Engine, chain consensus.ChainHeaderReader,
// 헤더 검증
if err := engine.VerifyHeader(chain, block.Header()); err != nil {
return fmt.Errorf("header verification failed: %v", err)
}
// Uncle 검증
if err := engine.VerifyUncles(chain, block); err != nil {
return fmt.Errorf("uncle verification failed: %v", err)
}
return nil
}
```

2. 새 블록 준비

```
func prepareBlock(engine consensus.Engine, chain consensus.ChainHeaderReader, h
// 합의 엔진별 헤더 준비
if err := engine.Prepare(chain, header); err != nil {
return fmt.Errorf("block preparation failed: %v", err)
}
return nil
}
```

3. 블록 완료 및 조립

```
func finalizeBlock(engine consensus.Engine, chain consensus.ChainHeaderReader,
// 블록 완료 및 조립
block, err := engine.FinalizeAndAssemble(chain, header, state, body, receipts)
if err != nil {
return nil, fmt.Errorf("block finalization failed: %v", err)
}
return block, nil
}
```

에러 처리 메커니즘

1. 공통 합의 에러

```
var (
ErrUnknownAncestor = errors.New("unknown ancestor")
ErrPrunedAncestor = errors.New("pruned ancestor")
ErrFutureBlock = errors.New("block in the future")
ErrInvalidNumber = errors.New("invalid block number")
ErrInvalidTerminalBlock = errors.New("invalid terminal block")
)
```

2. Ethash 특정 에러

```
var (
errOlderBlockTime = errors.New("timestamp older than parent")
errTooManyUncles = errors.New("too many uncles")
errDuplicateUncle = errors.New("duplicate uncle")
errUncleIsAncestor = errors.New("uncle is ancestor")
errDanglingUncle = errors.New("uncle's parent is not ancestor")
)
```

3. Beacon 특정 에러

```
var (
```

```
errTooManyUncles = errors.New("too many uncles")

errInvalidNonce = errors.New("invalid nonce")

errInvalidUncleHash = errors.New("invalid uncle hash")

errInvalidTimestamp = errors.New("invalid timestamp")

)
```

관련 시스템

의존성 관계

1. **core/types**: 블록, 헤더, 트랜잭션 타입 정의
2. **params**: 네트워크 매개변수 및 포크 설정
3. **core/state**: 상태 관리 및 업데이트
4. **crypto**: 해시 함수 및 암호학적 연산
5. **common**: 공통 유틸리티 및 타입

상호작용 패턴

- **블록 생성**: 마이너/검증자가 새 블록 생성 시 합의 규칙 적용
- **블록 검증**: 수신된 블록이 합의 규칙을 준수하는지 검증
- **체인 선택**: 가장 긴 유효한 체인 선택
- **포크 해결**: 체인 분기 시 올바른 체인 선택

영향 범위

합의 알고리즘 검증 시스템의 변경은 다음 영역에 영향을 미칩니다:

1. **네트워크 보안**: 전체 네트워크의 보안 수준

2. **블록 생성**: 새 블록 생성 규칙 및 검증

3. **동기화**: 노드 간 블록체인 동기화

4. **포크 처리**: 하드포크 및 소프트포크 처리

5. **성능**: 블록 검증 속도 및 처리량

이 합의 알고리즘 검증 시스템은 Ethereum 네트워크의 보안과 무결성을 보장하는 핵심 구성요소로, 모든 블록이 네트워크 규칙을 준수하도록 보장합니다.